
Bases de Datos 2

— Administración y Seguridad —

Seguridad - Conceptos

Conceptos fundamentales:

- **Confidencialidad:** prevenir accesos no autorizados a los datos.
- **Integridad:** asegurar que los datos no sean alterados indebidamente.
- **Disponibilidad:** garantizar que los datos estén accesibles cuando se necesiten.
- **Seguridad en capas:** controles a nivel de red, sistema operativo, base de datos, aplicación.
- **Principio de privilegio mínimo:** cada usuario/sistema debe tener solo los permisos estrictamente necesarios.

Seguridad - Conceptos

- Seguridad Física: Protección del hardware y almacenamiento.
- Seguridad Lógica: Control de acceso a datos y operaciones.
- Autenticación → ¿Quién es el usuario?
- Autorización → ¿Qué puede hacer?
- Auditoría → ¿Qué hizo, quién y cuando?
- ❑ Política de Seguridad: Conjunto de reglas que define qué está permitido.
- ❑ Mecanismo de Seguridad: Implementación técnica que hace cumplir la política.

Aspectos del Problema de Seguridad

El ámbito de la seguridad de las bases de datos es amplio.

Abarca aspectos relacionados con hardware, software, personas y datos:

- **Legales, sociales y éticos**
 - ¿el solicitante tiene el derecho legal de obtener determinada información? (saldos de cuentas de clientes)
 - ¿cómo asegurar que no se revelen datos confidenciales a cambio de sobornos u otros favores?
- **Política gubernamental, institucional o corporativa**
 - ¿qué información no debe estar disponible al público? (historiales médicos...)
 - ¿cómo se decide 'quién' puede acceder a 'qué'?
- **Controles físicos**
 - ¿cómo proteger físicamente contra intrusos las salas en donde están los sistemas informáticos?

Aspectos del Problema de Seguridad

- Seguridad del Sistema Operativo
 - ¿borra el SO el contenido de áreas de almacenamiento y archivos de datos cuando no se necesitan?
 - ¿el SO permite el acceso directo a los ficheros de la base de datos?
 - si se usan contraseñas para el acceso al SO ¿cómo se mantienen en secreto?
 - ¿con qué frecuencia se cambian?
- Seguridad de la Red
 - La seguridad en el nivel de software de red es hoy en día fundamental, tanto en Internet como en las redes privadas de las organizaciones.
- **Aspectos específicos del Sistema de Bases de Datos (*)**
 - **¿dispone el sistema de BD del concepto de propiedad de la información?**
 - **¿cómo evitar que los usuarios puedan acceder a ciertas partes de la BD?**
 - **¿cómo limitar las operaciones que los usuarios pueden realizar sobre la BD?**

Hay que adoptar medidas de seguridad en todos estos niveles, pero nos centraremos en el último de ellos
(*)

Aspectos del Problema de Seguridad

El SGBD debe garantizar:

- **Integridad:** la información se encuentra protegida frente a modificaciones inadecuadas. La modificación de datos incluye la creación, inserción, modificación, cambio del estado de los datos y el borrado.
- **Disponibilidad:** que los objetos de la BD estén disponibles para un usuario humano o para un programa que tenga los derechos correspondientes.
- **Confidencialidad:** tiene relación con la protección de los datos frente al acceso no autorizado.

Amenazas a las bases de datos: generan la pérdida o la degradación de algunos de los objetivos de seguridad: integridad, disponibilidad y confidencialidad

Medidas de Control

Existen dos medidas de control principales que se utilizan para proporcionar seguridad a los datos:

- **Control de accesos:** creación de cuentas de usuario y contraseñas para controlar el proceso de entrada al sistema y determinar qué acciones puede realizar cada usuario.
- **Cifrado de datos:** Se utiliza para proteger datos confidenciales (por ejemplo, números de tarjetas de crédito) que se transmiten a través de redes de comunicación. También puede aplicarse dentro de la base de datos para brindar protección adicional a información sensible. Los datos se codifican mediante algoritmos de cifrado.

Seguridad de Acceso - Responsabilidad del DBa

Entre las responsabilidades del DBA se encuentran la concesión y retirada de privilegios a los usuarios que necesitan utilizar el sistema, y la clasificación de usuarios y datos en función de la política de la organización. Esto se conoce como control de acceso discrecional, y se basa en el establecimiento de privilegios.

El DBA posee una cuenta de administrador o superusuario, que le permite conceder o retirar privilegios a las cuentas o usuarios individuales o a los grupos de usuarios para ejecutar los siguientes tipos de usuarios:

- Creación de cuentas
- Concesión de privilegios
- Retirada de privilegios
- Asignación de un nivel de seguridad

Creación de Usuarios en PostgreSQL

USER vs ROLE: ¿Cuál usar?

CREATE USER

- Crea un usuario con LOGIN automático
- Puede conectarse directamente
- Ideal para: personas reales

CREATE ROLE

- Grupo de permisos sin LOGIN por defecto

-- Crear un usuario con LOGIN

```
CREATE USER juan  
WITH PASSWORD 'clave_segura';
```

-- Crear un rol (grupo de permisos)

```
CREATE ROLE vendedor;
```

-- Asignar rol a usuario

```
GRANT vendedor TO juan;
```

💡 **En PostgreSQL:** CREATE USER = CREATE ROLE + LOGIN
Internamente son lo mismo, pero USER permite conexión directa por defecto

Privilegios

Un privilegio es un **derecho de acceso** o **autorización** para realizar determinada **operación** sobre **ciertos objetos** de la BD

Un **usuario** puede tener **diversos privilegios** sobre **distintos objetos**.

- Ej: privilegio SELECT sobre empleados, departamentos y proyectos

Diferentes usuarios pueden tener **privilegios distintos** sobre un mismo objeto

- U1 con privilegios SELECT, INSERT y DELETE sobre empleados
- U2 con privilegio SELECT sobre empleados

Tipos de privilegios

- Privilegios de **nivel de cuenta**
- Privilegios de **nivel de objeto** de la base de datos

Privilegio

- **Privilegios a Nivel de Cuenta (o de Sistema):**

- **S** Qué controlan: La capacidad de crear, alterar o eliminar "contenedores" de datos y gestionar el acceso de otros.
- Analogía: Es como tener las llaves de un edificio. Deciden si puedes construir una nueva habitación, invitar a alguien a entrar o demoler una pared.

- **Privilegios a Nivel de Objeto:**

- Qué controlan: La manipulación de los datos dentro de ese objeto.
- Analogía: Es como tener acceso a un cajón específico dentro de una oficina. Quizás puedes ver los archivos que hay dentro (SELECT), pero no tienes permiso para borrarlos o escribir en ellos

Privilegios a Nivel de Cuenta

Son los privilegios particulares de cada usuario, independiente de los objetos de la BD

Tipos de privilegios

- CREATE SCHEMA, DROP SCHEMA
- CREATE TABLE, ALTER TABLE, DROP TABLE
- CREATE VIEW, DROP VIEW
- SELECT, INSERT, UPDATE, DELETE (sobre cualquier tabla)

No están definidos en el estándar SQL, sino que son específicos de cada SGBD

Privilegios a Nivel de Objeto

Permiten especificar qué usuarios tienen qué privilegios sobre qué objetos concretos.

Están definidos en el estándar SQL.

Propietario de un objeto de base de datos:

- El usuario que lo ha creado (= cuenta de BD en la que fue creado)...
- posee todos los privilegios (posibles) sobre el objeto,...
- y capacidad de conceder tales privilegios a otros usuarios (GRANT)...
- e incluso propagar dicha capacidad de concesión (GRANT OPTION)

Cancelación de Privilegios (Seguridad de Autorización)

Cancela los privilegios indicados a determinado usuario.

En SQL estándar y PostgreSQL:

REVOKE [**GRANT OPTION FOR**] <privilegios> **ON** <objetos> **FROM** <sujetos>

Ejemplo: **REVOKE INSERT ON** productos **FROM** 'pedro';

REVOKE SELECT ON fotografos **FROM** 'julia';

REVOKE UPDATE ON editoriales **FROM** 'julia', 'ruben';

Tipos de Privilegios (de Nivel de Objeto)

- **SELECT**: Permite consultar datos de una tabla o vista. Puede otorgarse sobre **toda la tabla** o **sólo sobre columnas específicas**, lo que brinda control granular de acceso. Ej: `GRANT SELECT (dni, nombre) ON Clientes TO 'consulta';`
- **UPDATE**: sobre una tabla concreta, quizá de sólo algunas columnas. Ej: `GRANT UPDATE (saldo) ON Cuentas TO 'cajero1';`
- **INSERT**: autoriza la inserción de nuevas filas en una tabla concreta. Se puede especificar qué columnas pueden recibir valores (las demás deben tener valores por defecto o permitir NULL). Ej: `GRANT INSERT (numero_cuenta, saldo) ON Cuentas TO 'cajero2';`
- **DELETE**: sobre cierta tabla. Ej: `GRANT DELETE ON Movimientos TO 'admin_banco';`
- **REFERENCES**: permite establecer referencias a columnas concretas de cierta tabla mediante restricciones de integridad de cualquier tipo. Ej: `GRANT REFERENCES (numero_cuenta) ON Cuentas TO 'app_sistema';`
- **USAGE**: Es el privilegio más básico: permite la **existencia** de un usuario o rol en el sistema, pero **sin permisos adicionales** sobre objetos ni bases de datos, es decir, sólo le permite al usuario conectarse al servidor. Se usa principalmente para crear usuarios con acceso restringido, o para roles que recibirán permisos más adelante. Ej: `GRANT USAGE ON *.* TO 'usuario';`
- **ALL PRIVILEGES**: todos los privilegios que posee el usuario que concede (el que ejecuta GRANT). Ej: `GRANT ALL PRIVILEGES ON banco * TO 'gerente1';`

Modelo de Matriz de Acceso (DAC)

$M[\text{usuario}, \text{objeto}] = \text{conjunto de privilegios}$

USUARIOS (Sujetos) →

Matriz M	admin	juan	maria	invitado
empleados	SELECT, INSERT UPDATE, DELETE	SELECT	SELECT, UPDATE	Ø
ventas	TODOS	SELECT, INSERT	SELECT	Ø
clientes	TODOS	SELECT	SELECT, UPDATE	SELECT (solo suyos)

🔺 Estructura de la Matriz de Acceso:

- **Filas:** Objetos (tablas, vistas, procedimientos)
- **Columnas:** Sujetos/Usuarios
- **Celdas:** Privilegios que tiene cada usuario sobre cada objeto

Control de Acceso Discrecional

Para formalizar la seguridad, debemos entender que el Control de Acceso Discrecional (DAC) se basa en el modelo de Matriz de Acceso. En este modelo, definimos tres elementos clave:

- **Sujetos (S):** Son los entes activos que solicitan acceso (usuarios, cuentas o programas).
- **Objetos (O):** Son los entes pasivos que contienen la información (relaciones, vistas, columnas o registros).
- **Derechos (R):** Son los tipos de operaciones permitidas (lectura, inserción, borrado, actualización). La matriz $M(S, O)$ representa el conjunto de derechos que el sujeto S tiene sobre el objeto O . El estándar SQL implementa este modelo de matriz como una realización práctica, donde las filas son los usuarios y las columnas son las tablas o vistas.

Formalización del Dac

En el DAC (Control de Acceso Discrecional), el propietario de un objeto (generalmente quien lo creó) tiene la potestad absoluta de decidir quién más puede acceder a él.

- Operaciones: La matriz se gestiona dinámicamente mediante los comandos GRANT (que agrega entradas a la matriz) y REVOKE (que las elimina).
- Delegación: La propagación de privilegios crea un grafo de delegación donde un usuario con la opción GRANT OPTION puede actuar como un emisor secundario de privilegios.

Aunque es el modelo más flexible, su vulnerabilidad radica en la complejidad de rastrear propagaciones profundas de privilegios.

Formalización del Dac

Sea:

- S = conjunto de sujetos
- O = conjunto de objetos
- R = conjunto de derechos

Se define una matriz M tal que: $M(s, o) \subseteq R$

Operaciones sobre la matriz

- GRANT $\rightarrow$ agrega derechos a $M(s,o)$
- REVOKE $\rightarrow$ elimina derechos de $M(s,o)$

Delegación: los privilegios pueden propagarse mediante un grafo dirigido de concesiones.

Limitaciones del Dac

El DAC tiene fallas estructurales:

1. Dependencia del Propietario: La seguridad es descentralizada; si un propietario es descuidado, la seguridad de sus datos se compromete.
2. Fuga Indirecta de Información: El DAC no impide que un usuario autorizado a leer un dato copie esa información en una tabla propia y la comparta con terceros no autorizados.
3. Gestión en Grandes Organizaciones: Administrar miles de usuarios y tablas mediante una matriz individual es ineficiente y propenso a errores..

Propagación de Privilegios

- Cuando u1, propietario de una tabla R, concede privilegios sobre R a otro usuario u2, también puede darle la opción de otorgar dichos privilegios a terceros.
 - `GRANT SELECT, INSERT ON R TO u2 WITH GRANT OPTION;`
- A partir de ese momento, u2 podrá conceder esos privilegios sobre R a otras cuentas de usuario.
 - propagación de privilegios
- El SGBD mantiene la pista de la concesión y propagación de privilegios entre los usuarios

Propagación de Privilegios

- Revocar la GRANT OPTION de cierto privilegio a u2 no le cancela dicho privilegio, pero evita que u2 lo propague...
 - REVOKE **GRANT OPTION FOR INSERT** ON R FROM u2;
- Pero si u1 sí revoca el privilegio concedido a u2, ¿qué ocurre con los que u2 ya propagó a partir de la concesión? ...
 - Privilegios abandonados (no aplicables)
- Y si el propietario de una vista V pierde el privilegio SELECT sobre alguna de las tablas base de V...
 - Vista abandonada (no válida, no puede utilizarse)

PropagaciónDe Privilegios

Cuando un usuario revoca privilegios, indica una de estas opciones...

CASCADE

- el SGBD revocará automáticamente todos los privilegios que quedarían abandonados y
- eliminará las vistas que quedarían abandonadas.

RESTRICT

- el SGBD no revocará un privilegio si ello dejara privilegios o vistas abandonados.
- es la opción por omisión.

Ejemplos:

REVOKE SELECT ON Fotografia FROM julia CASCADE;

Propagación de Privilegios

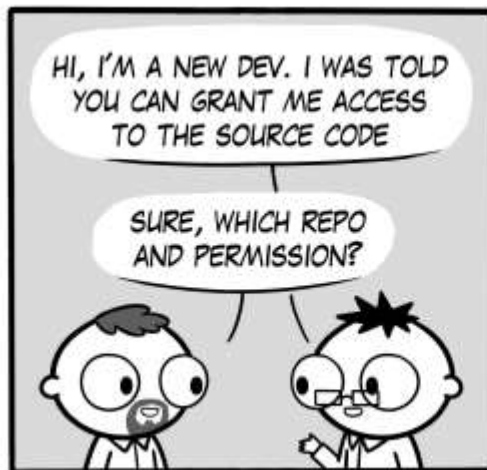
- La **revocación completa y correcta** de privilegios sólo se conseguirá si el SGBD sigue la pista de la concesión y propagación de privilegios.
- Un usuario puede recibir un mismo privilegio desde varios usuarios. Sólo lo perderá si se lo revocan todos ellos.
- **Eliminar un objeto** (dominio, tabla, columna o vista), **revoca automáticamente todo privilegio sobre el objeto eliminado, en todos los usuarios.**

Tipos de privilegios

- Privilegios Directos → otorgados explícitamente.
- Privilegios Heredados → recibidos por delegación.

Problema en sistemas grandes

- La revocación puede requerir recalcular múltiples dependencias.



Control de Acceso Basado en Roles (rbac)

El RBAC surgió para solucionar la rigidez del DAC en entornos empresariales. En lugar de asignar permisos a usuarios, los permisos se asocian a Roles (que representan funciones laborales) y los usuarios se asignan a estos roles.

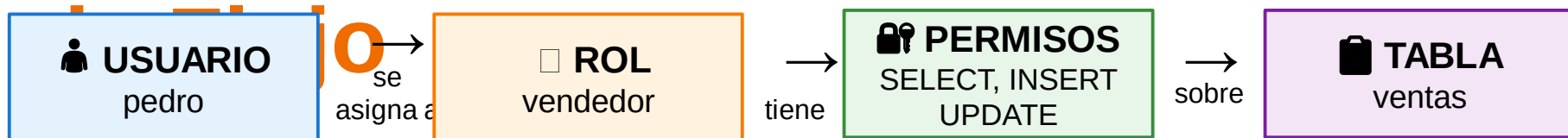
El modelo se define mediante cuatro entidades:

1. **Usuario:** El individuo humano.
2. **Rol:** La descripción de una función (ej. "Contador", "Analista").
3. **Permisos:** Privilegios sobre objetos de la BD.
4. **Sesión:** La activación temporal de un subconjunto de roles por parte de un usuario. Beneficio: RBAC reduce drásticamente la complejidad de la matriz de acceso al agrupar cientos de permisos en un solo rol.

Jerarquía de Roles

Los roles pueden organizarse en una jerarquía donde los roles superiores heredan automáticamente los permisos de los inferiores. Por ejemplo, el rol "Administrador" puede incluir todos los privilegios del rol "Usuario" más privilegios de gestión. Esto permite una escalabilidad organizacional que el DAC no puede ofrecer, convirtiendo al RBAC en el modelo dominante en sistemas corporativos modernos.

RBAC en PostgreSQL - Diagrama



-- 1. Crear el rol:

```
CREATE ROLE vendedor;
```

-- 2. Asignar permisos al rol (Rol → Permisos → Tabla)

```
GRANT SELECT, INSERT, UPDATE ON ventas TO vendedor;
```

-- 3. Asignar rol al usuario (Usuario → Rol)

```
GRANT vendedor TO pedro;
```

💡 Ventaja de RBAC:

• Sin RBAC (DAC tradicional):

50 permisos × 100 usuarios = 5,000 sentencias GRANT

• Con RBAC:

50 permisos → 1 rol → 100 usuarios = solo 150 sentencias

¡97% menos de trabajo administrativo!

Control de Acceso Basado en Roles

- (RBAC) RBAC0: Modelo básico.
- RBAC1: Jerarquía de roles (herencia).
- RBAC2: Restricciones (Separación de funciones).
- RBAC3: Modelo completo (jerarquía + restricciones).

Separación de Funciones (SoD)

Un mismo usuario no debe:

- Crear un pago
- Autorizar el mismo pago

RBAC no es solo agrupar permisos, es un modelo formal con jerarquías y restricciones organizacionales

Seguridad Multinivel -Control de Acceso Obligatorio (mac)

A diferencia del DAC, en el MAC el acceso no depende de la voluntad del propietario, sino de una política central obligatoria. Se basa en la clasificación de los sujetos y los objetos en niveles jerárquicos de seguridad:

- Top Secret (TS)
- Secret (S)
- Confidential (C)
- Unclassified (U)

Seguridad Multinivel - Modelo Bell - Lapadula

Este es el modelo formal más utilizado en entornos de alta seguridad (militar o gubernamental). Define dos reglas cruciales:

1. Propiedad de Seguridad Simple (No Read Up): Un sujeto solo puede leer objetos de su nivel o inferiores.
2. Propiedad Estrella (*) (No Write Down): Un sujeto solo puede escribir en objetos de su nivel o superiores. Esta propiedad estrella es fundamental: evita que un usuario con nivel "Secreto" baje información confidencial a una tabla "No Clasificada".

Seguridad Multinivel - Comparación Dac Vs. Mac

Característica	DAC	MAC
Control	Propietario	Política central
Flexibilidad	Alta	Baja
Aplicación típica	Empresas	Sistemas militares
Delegación	Permitida	No permitida

Seguridad Multinivel -Relaciones

Multinivel

En MAC, el esquema de la relación se expande para incluir la clasificación.

Cada tupla (o incluso cada celda) tiene un atributo de clasificación asociado.

Un esquema típico sería: **R(Atributo1, Clase1, Atributo2, Clase2, ..., ClaseTupla)**

Seguridad Multinivel -Poliinstanciación

Es el concepto más avanzado de MAC. Ocurre cuando existen múltiples versiones de la misma tupla con diferentes niveles de seguridad.

Ejemplo: Si un usuario de nivel U intenta insertar un empleado que ya existe en nivel S, el sistema le permite hacerlo creando una versión nivel U para evitar revelar que ya existe información clasificada (lo cual sería un canal oculto de inferencia).

Es decir, resuelve el problema de inferencia a costa de la integridad, al evitar que un usuario de nivel inferior detecte la existencia de datos clasificados.

Seguridad en Bases de Datos

Estadísticas

El problema principal acá es la **inferencia**.

Un atacante puede deducir datos confidenciales individuales mediante consultas agregadas (SUM, AVG) si la población seleccionada es muy pequeña.

La inferencia puede ocurrir sin violar ningún privilegio explícito.

Ejemplo:

- Query1 = AVG salarios depto X
 - Query2 = AVG salarios depto X excluyendo empleado A
- **La diferencia revela salario de A.**

Mitigación en Bases de Datos Estadísticas

Para prevenir la inferencia, se puede aplicar:

- **Umbral mínimo:** No mostrar resultados si la población es menor a n registros.
- **Supresión de resultados:** Ocultar celdas sensibles.
- **Ruido Estadístico:** Introducir ligeras imperfecciones en los resultados para dificultar la deducción exacta.

Uso de Vistas Como Mecanismo de Seguridad

El usuario u1 es propietario de la tabla R(a1,a2,a3,a4,a5)

- Si u1 desea que otro usuario u2 pueda leer sólo algunas columnas a1,a2,a3 de R

```
CREATE VIEW vista_personalizada AS SELECT a1, a2, a3 FROM R;
```

```
GRANT SELECT ON vista_personalizada TO u2;
```

- Si u1 desea que u2 lea sólo algunas filas de R, las que satisfacen cierta condición Q

```
CREATE VIEW vista_personalizada_filtrada AS SELECT * FROM R WHERE Q;
```

```
GRANT SELECT ON vista_personalizada_filtrada TO u2;
```

Para poder crear una vista, el usuario debe poseer el privilegio SELECT sobre cada tabla base de la vista, además del privilegio de cuenta CREATE VIEW

Otros Aspectos de Seguridad: Auditoría

El seguimiento de auditoría permite ...

- Verificar que todo está en orden,
- Descubrir si alguien ha accedido de forma ilegal, o ha realizado operaciones sin autorización.

Cada entrada en la tabla de auditoría podría contener...

- puesto o terminal desde la que se invocó la operación,
- usuario que solicitó la operación,
- operación realizada (por ejemplo UPDATE),
- objetos afectados: base de datos, tablas (ficheros), filas (registros)
- y columnas (campos),
- fecha y hora de la operación.

Además de ayudar a garantizar la Integridad (sabiendo quién modificó qué y cuándo) y a verificar la Confidencialidad (detectando accesos no autorizados), en muchos casos el hecho de mantener un seguimiento de auditoría alcanza para desanimar a posibles “espías”.

Auditoría - Tipos

Podemos diferenciarla en:

- **Auditoría a nivel BD:** Registra operaciones físicas de bajo nivel (actualización de tuplas).
- **Auditoría a nivel Aplicación:** Registra acciones lógicas de negocio (ej. "Aprobación de crédito"). Limitación: El log nativo a menudo es insuficiente porque no rastrea al usuario final de la aplicación, solo a la cuenta técnica de conexión.

Estrategias de Auditoría en Postgresql

Existen principalmente dos enfoques para implementar auditoría:

1. Auditoría de Datos (Vía Triggers/Disparadores):

- ¿Qué hace? Captura los cambios específicos en las filas (INSERT, UPDATE, DELETE).
- ¿Cómo? Se crea una tabla espejo o de histórico y una función automática que guarda el "antes" y el "después".
- Ventaja: Muy granular. Permite consultar el historial fácilmente con SQL.
- Desventaja: Puede impactar levemente el rendimiento en escrituras masivas.

2. Auditoría de Actividad (Vía Logs/Extensiones):

- ¿Qué hace? Registra qué sentencias SQL se ejecutaron.
- ¿Cómo? Configurando postgresql.conf o usando la extensión pgaudit.

Ejemplo Práctico - Tabla de Auditoría

Vamos a crear una tabla centralizada para registrar cambios. Usaremos el tipo de dato JSONB de PostgreSQL para ser flexibles y guardar cualquier estructura de tabla.

```
CREATE TABLE auditoria_log (  
  id SERIAL PRIMARY KEY,  
  esquema TEXT NOT NULL,  
  tabla TEXT NOT NULL,  
  usuario TEXT NOT NULL,      -- Quién hizo el cambio  
  accion TEXT NOT NULL,      -- INSERT, UPDATE o DELETE  
  datos_viejos JSONB,        -- Valor antes del cambio  
  datos_nuevos JSONB,        -- Valor después del cambio  
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Ejemplo Práctico - Función para los

Triggers

```
CREATE OR REPLACE FUNCTION fn_auditoria() RETURNS TRIGGER AS $$  
BEGIN
```

```
    IF (TG_OP = 'INSERT') THEN
```

```
        INSERT INTO auditoria_log (esquema, tabla, usuario, accion, datos_nuevos)
```

```
        VALUES (TG_TABLE_SCHEMA, TG_TABLE_NAME, session_user, 'INSERT', to_jsonb(NEW));
```

```
        RETURN NEW;
```

```
    ELSIF (TG_OP = 'UPDATE') THEN
```

```
        INSERT INTO auditoria_log (esquema, tabla, usuario, accion, datos_viejos, datos_nuevos)
```

```
        VALUES (TG_TABLE_SCHEMA, TG_TABLE_NAME, session_user, 'UPDATE', to_jsonb(OLD), to_jsonb(NEW));
```

```
        RETURN NEW;
```

```
    ELSIF (TG_OP = 'DELETE') THEN
```

```
        INSERT INTO auditoria_log (esquema, tabla, usuario, accion, datos_viejos)
```

```
        VALUES (TG_TABLE_SCHEMA, TG_TABLE_NAME, session_user, 'DELETE', to_jsonb(OLD));
```

```
        RETURN OLD;
```

```
    END IF;
```

```
    RETURN NULL;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

Ejemplo Práctico - Crear Triggers en las Tablas Críticas

-- Vinculamos la función a la tabla que queremos vigilar

```
CREATE TRIGGER trg_auditoria_empleados  
AFTER INSERT OR UPDATE OR DELETE ON empleados  
FOR EACH ROW EXECUTE FUNCTION fn_auditoria();
```

Auditoría Avanzada - Pgaudit

Para entornos corporativos donde se requiere auditoría forense (no solo cambios de datos, sino también quién leyó datos o intentos fallidos de acceso), los triggers no son suficientes.

Extensión pgAudit:

- Es el estándar de la industria para PostgreSQL.
- Registra a nivel de motor, imposible de evadir por usuarios normales.
- Permite clasificar qué auditar: READ (SELECT), WRITE (INSERT/UPDATE), DDL (CREATE/DROP), ROLE (GRANT/REVOKE).

Configuración en postgresql.conf:

```
shared_preload_libraries = 'pgaudit'
```

```
pgaudit.log = 'write, ddl, role'
```

```
pgaudit.log_catalog = 'off'
```

Cifrado

El **cifrado** es una técnica fundamental de **seguridad de la información** que permite proteger los datos frente a accesos no autorizados. Consiste en transformar los datos originales (texto plano) en un formato ilegible (texto cifrado) mediante un **algoritmo de cifrado** y una **clave**.

Solo quien posea la **clave de descifrado** podrá revertir el proceso y recuperar los datos originales. De esta forma, aunque los datos sean interceptados o robados, no podrán interpretarse sin la clave correspondiente.

Sirve para proteger datos confidenciales:

- Transmitidos por cualquier tipo de red
- Almacenados en la BD (para proteger áreas de la BD)

Componentes del Cifrado

- Texto plano (plaintext): Es la información original que se desea proteger.
- Algoritmo de cifrado: Es la función matemática que transforma el texto plano en texto cifrado. Ejemplo: AES, RSA, DES.
- Clave de cifrado: Valor secreto utilizado por el algoritmo para realizar la conversión. Su longitud y complejidad determinan el nivel de seguridad.
- Texto cifrado (ciphertext): Es el resultado ilegible del proceso de cifrado.
- Descifrado (decryption): Proceso inverso que recupera el texto plano original utilizando la clave correspondiente.

Tipos de Cifrado

Existen dos grandes tipos de cifrado, según cómo se manejan las claves: simétrico y asimétrico.

1. Cifrado simétrico

- Utiliza **la misma clave** para cifrar y descifrar.
- Es rápido y eficiente, ideal para grandes volúmenes de datos.
- Requiere que la clave se comparta de forma segura entre las partes.
- Ejemplos: AES (Advanced Encryption Standard), DES, 3DES, Blowfish.

Tipos de Cifrado

2. Cifrado asimétrico

- Usa un **par de claves diferentes: una pública** (para cifrar) y **una privada** (para descifrar).
- Es más seguro, ya que la clave privada no se comparte, pero más lento que el simétrico.
- El cifrado de los datos reduce el rendimiento del sistema.
- El SGBD debe decodificar/codificar los datos manejados.
- Ejemplos: RSA, ECC (Elliptic Curve Cryptography).

Cifrado

El cifrado puede aplicarse en diferentes niveles dentro de un sistema de base de datos:

- Cifrado a nivel de aplicación:
 - Los datos se cifran antes de ser enviados a la base de datos.
 - Ventaja: los datos permanecen cifrados incluso si la base es comprometida.
 - Desventaja: mayor complejidad en el código de la aplicación.
- Cifrado a nivel de columna o campo:
 - Se cifran columnas específicas que contienen información sensible (como contraseñas, DNI, tarjetas, etc.).
 - Ejemplo: `pgp_sym_encrypt()` y `pgp_sym_decrypt()` de `pgcrypto`
- Cifrado a nivel de disco o tablespace (TDE – Transparent Data Encryption):
 - Cifra automáticamente los datos almacenados en disco sin necesidad de modificar las consultas.
 - Implementado en sistemas como Oracle, SQL Server y PostgreSQL Enterprise Edition.
 - El usuario accede a los datos descifrados de forma transparente.

Cifrado en PostgreSQL - pgcrypto

⚠️ Primero instalar la extensión: `CREATE EXTENSION IF NOT EXISTS pgcrypto;`

```
-- Cifrar y guardar (pgp_sym_encrypt)
INSERT INTO secretos (id, dato_cifrado)
VALUES (1, pgp_sym_encrypt('texto_secreto', 'mi_clave'));
-- Descifrar (pgp_sym_decrypt)
SELECT pgp_sym_decrypt(dato_cifrado, 'mi_clave') AS texto_plano
FROM secretos WHERE id = 1;
```

MySQL:

`AES_ENCRYPT()` y `AES_DECRYPT()`

Son funciones exclusivas de MySQL

PostgreSQL:

Usar `pgp_sym_encrypt()` y `pgp_sym_decrypt()`
de la extensión `pgcrypto`

Cifrado en PostgreSQL - Funciones de Hash

Para contraseñas o comprobaciones de integridad se usan funciones hash, que no son reversibles:

```
SELECT SHA2('mi_contraseña', 256);  
SELECT MD5('mi_contraseña');
```

Estas funciones no permiten descifrar el valor original, solo verificar coincidencias.

* Nota: MD5 se considera obsoleto y no seguro para almacenar contraseñas debido a su vulnerabilidad a colisiones. Se incluye como ejemplo de función hash, pero se debe preferir SHA2 o, idealmente, algoritmos de hashing de contraseñas como bcrypt o Argon2 a nivel de aplicación.

Sql Injection(Inyección de Sql)

La inyección de SQL es una amenaza común en las Bases de Datos.

Los programas y aplicaciones web que acceden a una base de datos pueden enviar comandos y datos a esta, así como mostrar los datos recuperados de ella a través del navegador web.

En un **ataque de inyección SQL**, el atacante inyecta una cadena de entrada a través de la aplicación, lo que modifica o manipula la sentencia SQL en su beneficio.

Un ataque de inyección SQL puede dañar la base de datos de diversas maneras, como la manipulación no autorizada de la misma o la recuperación de datos confidenciales. También puede utilizarse para ejecutar comandos a nivel de sistema que pueden provocar que el sistema deniegue el servicio a la aplicación.

Sql Injection - Manipulación del Sql

Es el tipo más común de ataque de inyección. Modifica un comando SQL en la aplicación: por ejemplo, añadiendo condiciones a la cláusula WHERE de una consulta o expandiendo una consulta con componentes adicionales mediante operaciones de conjunto como UNION.

También son posibles otros tipos de ataques de manipulación. Un ataque de manipulación típico ocurre durante el inicio de sesión en la base de datos.

Cómo ocurre:

- La aplicación construye sentencias SQL concatenando texto + valores de entrada (sin sanitizar).
- Un atacante suministra valores que cambian la estructura de la consulta (por ejemplo, añadiendo condiciones lógicas o operadores).

Por ejemplo, supongamos que un procedimiento de autenticación simple emite la siguiente consulta y comprueba si se devolvió alguna fila:

```
SELECT * FROM users WHERE username = 'john' AND PASSWORD = 'johnspasswd' ;
```

El atacante podría intentar manipular la sentencia cambiándola por la siguiente:

```
SELECT * FROM users WHERE username = 'john' AND (PASSWORD = 'johnspasswd' OR 'x'='x');
```

SQL Injection: Comparación de Código

✗ CÓDIGO VULNERABLE

(Concatenación directa)

```
-- Entrada atacante:  
password = "" OR '1'='1' --"
```

```
const query =  
  "SELECT * FROM users  
  WHERE username="" +  
    username +  
    "" AND password="" +  
    password +  
    """,  
  ,
```

✓ CÓDIGO SEGURO

(Consulta parametrizada)

```
-- Parámetros separados:  
username = "" OR '1'='1' --"
```

```
const query =  
  "SELECT * FROM users  
  WHERE username = $1  
  AND password = $2
```

```
db.query(query, [user, pass])
```

💡 ¿Qué sucede?

✗ Vulnerable: La entrada se interpola → Se ejecuta como código SQL

✓ Seguro: Los parámetros se tratan como datos → No se ejecutan

Inyección de Código Malicioso

✗ ATAQUE: Inyección DROP TABLE

Entrada del atacante:

producto = ""; **DROP TABLE** ventas; --"

-- Código vulnerable:

```
INSERT INTO log  
VALUES (" + producto + ")"
```

⚠ RESULTADO:

Se ejecuta: INSERT ...

+ DROP TABLE ventas

→ ¡Base de datos destruida!

✓ PROTECCIÓN: Parametrización

Misma entrada atacante:

producto = ""; DROP TABLE..."

-- Código seguro:

```
INSERT INTO log  
VALUES ($1)
```

✓ RESULTADO:

Se inserta como texto:

""; DROP TABLE ventas; --"

→ Solo datos, no comando

🔒 REGLA DE ORO:

Nunca concatenes datos de usuario directamente en SQL.

Usa siempre consultas parametrizadas.

Sql Injection -InyecciónDe Llamadas a Funciones

En este tipo de ataque, se inserta una función de base de datos o una llamada a función del sistema operativo en una sentencia SQL vulnerable para manipular los datos o realizar una llamada privilegiada al sistema. Por ejemplo, es posible explotar una función que realiza algún aspecto relacionado con la comunicación de red. Además, las funciones contenidas en un paquete de base de datos personalizado, o cualquier función de base de datos personalizada, pueden ejecutarse como parte de una consulta SQL. En particular, las consultas SQL creadas dinámicamente pueden explotarse, ya que se construyen en tiempo de ejecución.

Sql Injection - Inyección de Llamadas a Funciones

Inyección de llamadas a funciones: El atacante inserta una llamada a una función de base de datos (p. ej. SLEEP(), LOAD_FILE(), INTO OUTFILE, etc.) para provocar comportamientos no deseados: DoS, extracción de ficheros o ejecución de código mediante funciones definidas.

- Ejemplo 1: DoS por SLEEP(): (detiene la ejecución del servidor durante una cantidad de segundos.

Entrada del atacante: ' ; **SELECT SLEEP(10000); --** '

Consulta vulnerable concatenada ejecuta SLEEP y retrasa el servidor → posible denegación de servicio.

- Ejemplo 2: obtener información

Entrada del atacante: “ ; **SELECT * FROM usuarios WHERE id = 1 AND IF(SUBSTRING(password, 1, 1) = 'a', SLEEP(5), 0); --** “

Si el primer carácter de la contraseña es 'a', la base de datos tardará 5 segundos en responder, confirmando la suposición del atacante.

- Ejemplo 3: lectura de archivos (condiciones de privilegios y funciones activas):

Entrada maliciosa: “ **OR 1=1 UNION SELECT LOAD_FILE('/etc/passwd') ; --** “

(Nota: LOAD_FILE() requiere permisos y que el archivo sea legible por el usuario del SGBD.)

Riesgos Asociados con Sql Injection

- **Determinación de la base de datos:** El atacante puede determinar el tipo de base de datos que se utiliza en el backend para poder usar ataques específicos que correspondan a las debilidades de un SGBD en particular.
- **Denegación de servicio.** El atacante puede inundar el servidor con solicitudes, denegando así el servicio a usuarios válidos, o puede eliminar algunos datos.
- **Evitar la autenticación.** Este es uno de los riesgos más comunes, en el que el atacante puede acceder a la base de datos como usuario autorizado y realizar todas las tareas deseadas.
- **Identificar parámetros inyectables.** En este tipo de ataque, el atacante recopila información importante sobre el tipo y la estructura de la base de datos backend de una aplicación web. Este ataque es posible gracias a que la página de error predeterminada que devuelven los servidores de aplicaciones suele ser excesivamente descriptiva.
- **Ejecución de comandos remotos.** Esto proporciona a los atacantes una herramienta para ejecutar comandos arbitrarios en la base de datos. Por ejemplo, un usuario remoto puede ejecutar procedimientos y funciones de bases de datos almacenadas desde una interfaz interactiva SQL remota.
- **Escalada de privilegios.** Este tipo de ataque aprovecha las fallas lógicas de la base de datos para aumentar el nivel de acceso.

Mitigación de Sqli

Mitigación Primaria (la más importante):

- Usar **consultas parametrizadas / prepared statements**
- Evitar la construcción manual de SQL con concatenación

Buenas prácticas adicionales para disminuir las posibilidades de sufrir SQLi:

- Validación / sanitización de entradas (whitelisting)
- Privilegios mínimos en cuentas de DB
- Manejo de errores genérico (no mostrar trazas)
- Uso de ORMs o APIs con protección contra inyección
- Auditoría y alertas sobre consultas sospechosas

Mitigación de Riesgos - Consultas Parametrizadas

Una **consulta parametrizada** (o **prepared statement**) es una sentencia SQL en la que **la estructura de la consulta y los valores de los datos se envían por separado** al motor de base de datos.

En lugar de construir la cadena SQL concatenando texto y variables (lo que permitiría inyectar código), la aplicación **define primero el comando SQL con marcadores de posición** (parámetros) **y luego envía los valores por separado**.

Consultas Parametrizadas - Funcionamiento P.1

Cuando se usa una consulta parametrizada, el proceso en el SGBD ocurre en dos fases separadas:

- Fase 1: Preparación

La aplicación envía al SGBD la estructura de la consulta con parámetros sin valores:

```
PREPARE stmt FROM 'SELECT * FROM usuarios WHERE nombre = ? AND clave = ?';
```

En esta fase:

- El servidor analiza y compila la consulta.
- Define su plan de ejecución.
- Los signos ? son lugares reservados para valores futuros.
- El motor no ejecuta la consulta todavía.

Consultas Parametrizadas - Funcionamiento P.2

Fase 2 – Ejecución

Luego, la aplicación envía los valores que reemplazan los parámetros:

```
EXECUTE stmt USING @nombre, @clave;
```

En esta fase ocurre algo clave:

- Los valores se tratan **exclusivamente como datos**, nunca como parte del código SQL.
- El motor **ya tiene definida la estructura** y sabe que los valores no pueden alterar su sintaxis.
- Si el valor contiene comillas, operadores (OR, ;, --, etc.), el motor **no los interpreta como SQL, sino como texto literal**.

HI, THIS IS
YOUR SON'S SCHOOL.
WE'RE HAVING SOME
COMPUTER TROUBLE.



OH, DEAR - DID HE
BREAK SOMETHING?
IN A WAY-



DID YOU REALLY
NAME YOUR SON
Robert'); DROP
TABLE Students;-- ?



OH. YES. LITTLE
BOBBY TABLES,
WE CALL HIM.

WELL, WE'VE LOST THIS
YEAR'S STUDENT RECORDS.
I HOPE YOU'RE HAPPY.



AND I HOPE
YOU'VE LEARNED
TO SANITIZE YOUR
DATABASE INPUTS.

Resumen

Seguridad en bases de datos no es solo gestión de permisos.

Es la integración de:

- Modelos formales (DAC, RBAC, MAC)
- Administración operativa (aplicación concreta de los modelos formales)
- Protección criptográfica (proteger la confidencialidad de los datos)
- Prevención de ataques (medidas para evitar vulnerabilidades explotables desde aplicaciones o usuarios maliciosos)
- Control de inferencia (mecanismos que impiden deducir información sensible a partir de consultas aparentemente legítimas)

Algoritmos y
estructura de datos



Gestión de
base de datos



Memes de programación

